Checkpoint 1 — Full English Speaking Script

Presenter: Trần Anh Dũng · 20226031

Slides: The current 13-slide HTML/PDF deck, including detailed framing and a separate task-allocation slide.

Target: 15 minutes of speaking; questions afterward. Read the speech blocks. Bracketed directions are cues, not spoken text. The speech is approximately 1,730 words: aim for about 125 words per minute plus diagram pauses. Rehearse once with a stopwatch;
the clock checkpoints account for short pauses and pointing at diagrams.

Slide	Topic	Duration	Clock
1	Introduction	35 sec	00:00–00:35
2	Objectives and features	65 sec	00:35–01:40
3	System modules and tools	75 sec	01:40–02:55
4	Message fields and types	65 sec	02:55–04:00
5	TCP framing and validation	65 sec	04:00–05:05
6	Requests, responses, and events	60 sec	05:05–06:05
7	Test and practice workflows	75 sec	06:05–07:20
8	Test message sequence	75 sec	07:20–08:35
9	Accounts and question model	75 sec	08:35–09:50
10	Tests, attempts, and answers	90 sec	09:50–11:20
11	Filters, history, and charts	70 sec	11:20–12:30
12	Task allocation	60 sec	12:30–13:30
13	Progress and next steps	90 sec	13:30–15:00

Slide 1 — Introduction · 00:00–00:35

Hello, teacher and everyone. My name is Trần Anh Dũng. Today I will present our first checkpoint for the Online Testing Application project.

Our group has four members: Trần Hữu Đức, Trần Anh Dũng, Trần Cao Phong, and Lê Diễm Hằng.

I will explain our required features, system architecture, protocol, data model, and implementation plan. We are currently at the design and environment setup stage. The diagrams describe our proposed system.

[Pause. Move on at 00:35.]

Slide 2 — Objectives and Features · 00:35–01:40

Our objective is to support timed test rooms and individual practice.

In Test Mode, a user creates a room and chooses the question count and duration. Others can view room status and join before the test starts. The owner controls when to start or end it.

Participants can change answers and submit early. At the deadline, the server finalizes unfinished attempts and calculates scores. Results are stored for later viewing.

Practice Mode is for one user with a chosen count and time limit. Shared features include registration, login, permissions, topic and difficulty filters, history, graphical statistics, and a GUI.

All these features are in our required scope. An important distinction is that one participant may finish while the room continues. Individual submission must not stop other participants from completing their tests.

[Point to the two modes. Target: 01:40.]

Slide 3 — System Modules and Tools · 01:40–02:55

The architecture has Tkinter clients, a C TCP server, and SQLite storage.

The server uses POSIX sockets, following the approach from our homework: socket, bind, listen, accept, send, and receive. The Python client connects directly over TCP and displays server-confirmed state.

Inside the server, the first module buffers the stream and dispatches complete messages. Account and session handlers establish identity. Room and attempt handlers manage participants and answers. A deadline scheduler and scoring service finalize results and save them before notifying users.

We plan to use pthreads for connections, a mutex for shared state, and separate send locks so responses and events cannot mix their bytes.

Our tools include GCC, cJSON, SQLite, libsodium for password hashing, and Docker Compose. Compose standardizes the Linux environment. The client performs network reads outside the Tkinter UI thread so waiting for the server does not freeze the interface.

[Point at the server modules, not only the three outer components. Target: 02:55.]

Slide 4 — Message Fields and Types · 02:55–04:00

This table describes our JSON message envelope.

The type field is a string identifying a command, a response, or an event. Request ID is a positive integer chosen by the client and echoed in the response. An event instead has an event name, because it is initiated by the server.

Responses include the boolean field ok. A successful response contains data; a failed response contains an error code and message. Clients use the code rather than trying to interpret the wording of an error.

The example requests to join room forty-one. The visible backslash-n represents one line-feed byte, hexadecimal zero-A. It is not two literal characters sent after the object.

This defines the envelope. Each command also has its own required payload fields, types, permissions, and response contract in our protocol document.

[Point to the table and the LF suffix. Target: 04:00.]

Slide 5 — TCP Framing and Validation · 04:00–05:05

TCP does not preserve application message boundaries. This example shows why the receiver needs a buffer.

The first receive call ends halfway through the command name. That fragment must be retained, not parsed immediately as invalid JSON.

The second receive call contains the remaining bytes of account-get, its line feed, and a complete logout request. The server can now extract two messages in order. Each is one JSON object ending at an LF byte.

After splitting, the server parses JSON and validates field types. Invalid JSON returns a protocol error for that line. A frame larger than sixty-four kibibytes is rejected and the connection closes.

Sending also requires a loop for partial writes. Question delivery is paginated, with at most five questions per page, to keep responses within the frame limit.

[Trace the unfinished command into the second chunk. Pause at each LF. Target: 05:05.]

Slide 6 — Requests, Responses, and Events · 05:05–06:05

The catalogue groups commands by responsibility.

Account commands handle registration and login. A successful login binds the user's identity to that connection. A request cannot provide another user ID to act as someone else.

Room commands create, list, join, start, and end rooms. Only the owner may start or end their room. Attempt commands fetch questions, save choices, and submit. They accept changes only for the user's own active attempt, before its deadline.

Every ordinary request receives a response with the same request ID. Server events are separate: for example, test-started and attempt-finished may arrive without a new client request.

This distinction lets an idle client learn that time has expired. History and statistics use ordinary authorized requests.

[Compare the request and event columns. Target: 06:05.]

Slide 7 — Test and Practice Workflows · 06:05–07:20

A room moves through three states: not started, ongoing, and finished.

When creating a room, the owner chooses count, duration, and filters, and also joins as a member. Others can join only before start. If too few questions match, the server reports an error instead of silently changing those filters.

Starting a room fixes the question set and order, creates one attempt per member, and sets a shared deadline.

During the test, a new choice replaces the previous choice for that question. Submitting early locks only the individual's attempt. The room finishes at the deadline, on owner-end, or when everyone has submitted.

For example, if one of three participants finishes a ten-minute test after six minutes, the others can continue to the original deadline.

Practice uses the same answer and scoring services, but with one participant and an immediate start after selecting settings.

[Emphasize submission versus room completion. Target: 07:20.]

Slide 8 — Test Message Sequence · 07:20–08:35

This simplified sequence connects the commands to database changes.

After room-start, the server checks ownership and room state, then saves the question selection, deadline, and attempts in one transaction. Participants are notified only after that succeeds.

Each receives its own attempt ID and requests question pages. The diagram groups notification and delivery for readability. Correct answer keys remain on the server.

For answer-set, the server validates the question and choice, commits the answer, then sends an acknowledgement. The GUI should show saved only after this ACK.

Submission scores and finalizes one attempt. Repeated submission returns its existing final result. Timeout uses the same finalization service.

Every write also checks server time, so scheduler delay cannot extend the test. After a disconnect, saved answers remain stored. Logging in again can retrieve the active attempt, but does not reset its deadline.

[Follow the arrows from start to result. Target: 08:35.]

Slide 9 — Accounts and Question Model · 08:35–09:50

Our first data-model diagram covers accounts and the question bank.

Users stores unique usernames and password hashes. Sessions stores login periods, expiry, and end reasons. A user can have many historical sessions, but only one active connection session at a time. Logout or disconnection closes it.

Each question belongs to a topic and has a difficulty level. Question-choices stores options A through D. A question's correct key references one of its own choices. The service checks that exactly four choices exist; a foreign key alone does not enforce that count.

Used question content stays immutable. A correction creates a new bank record, so old results still refer to the original question.

Password hashing protects stored passwords. It does not encrypt the TCP traffic. The initial deployment is intended for a controlled classroom network; we do not claim encrypted transport in the current design.

[Point first at sessions, then at the question relationships. Target: 09:50.]

Slide 10 — Tests, Attempts, and Answers · 09:50–11:20

The main distinction here is between a shared test and each person's attempt.

Tests stores settings and the shared deadline. Test-members records who joined. A waiting member has no attempt yet; attempts are created when the room starts.

For a concrete example, room forty-one has users seven and nine, with attempts eleven and twelve. User seven submits attempt eleven with eight correct answers out of ten. Attempt twelve stays active, so the room is still ongoing.

Test-questions fixes the selected set and order. Answers stores one current choice per attempt and question. Changing a choice updates that record.

Composite foreign keys ensure the answer belongs to the attempt's test, the question was selected for that test, and the choice exists for that question. Knowing another room's question ID does not grant access to answer it.

Time and permission checks still belong in the C service. Practice reuses the same tables with one member and one attempt. Filter and activity-log tables complete the schema, with detailed fields in the SQL file.

[Pause at attempts 11 and 12. Explain their different states. Target: 11:20.]

Slide 11 — Filters, History, and Charts · 11:20–12:30

Users customize tests by topic, difficulty, and question count. The server selects distinct matching questions; room participants receive the same set and order.

History stores mode, date, correct count, question count, and completion reason. Each user can access their own history and statistics.

The chart contains sample values: five, six, four, seven, and eight out of ten. That totals thirty correct answers from fifty questions, or sixty percent.

For different test lengths, progress uses percentages. Overall accuracy divides total correct answers by total questions. For example, eight out of ten plus ten out of twenty is eighteen out of thirty, also sixty percent.

Only finalized attempts count. Unanswered questions are incorrect. With no history, the GUI shows an empty state, not invented results. The illustrated chart is a design example, not collected user data.

[Point at the sample label. Target: 12:30.]

Slide 12 — Task Allocation · 12:30–13:30

We have divided implementation work into four areas.

I coordinate architecture, message contracts, integration, review, and the final demo. Đức handles C sockets and stream I/O, room and attempt states, deadlines, and scoring.

Phong handles SQLite, accounts, sessions, permission checks, and activity logging. Hằng handles the Python TCP client, Tkinter screens, history and statistics, and end-to-end tests.

The protocol and data-model documents are the shared contracts between these areas. This lets the GUI and server work proceed with agreed field names and behavior. Integration remains a group responsibility, especially for timing and multiple-client scenarios.

The next slide separates the design work already completed from application features that we still need to implement.

[Point to each member once; do not read every table entry. Target: 13:30.]

Slide 13 — Progress and Next Steps · 13:30–15:00

At this checkpoint, we have a separate repository, a Docker Compose development environment, functional-flow documents, an ERD, an executable SQL schema, and application-protocol contracts.

Twelve schema checks have passed. They verify relationships, constraints, and example scoring queries. We have also checked that the sample room records are consistent and that the illustrated receive chunks reconstruct the two intended JSON messages.

These are design and database validations, not end-to-end application tests. The C request handlers and GUI are not implemented yet.

Our first implementation step is TCP framing and a minimal request-response exchange. Next come accounts, sessions, the question bank, and shared attempt services. We then add room control, deadlines, GUI integration, history, and statistics.

Integration tests will cover multiple participants, partial reads and writes, answer changes near the deadline, early submission, and disconnection. These tests must confirm that server state and client displays remain consistent.

That concludes our proposed design and plan. We welcome your feedback before implementation. Thank you for listening.

[Finish around 15:00. Pause for questions.]

Rehearsal

- Use a stopwatch once and note each slide's finish time. The schedule totals exactly 900 seconds, but actual delivery depends on your speaking pace.
- Pause to point at the framing chunks, sequence arrows, and ERD relationships.
- If late, shorten the statistics example and table-reading on slides 11–12. Preserve the honest implementation status on slide 13.
- If early, slow down explanations and allow the audience time to inspect the diagrams. Do not add unsupported implementation claims.